UNIVERSIDAD EL BOSQUE
Programa de Ingeniería de Sistemas
Bases de Datos 2

PROPUESTA DE PROYECTO

SIRAV

Sistema Integral de Reservas y Alquiler de Vehículos

Plataforma web con PostgreSQL, MongoDB y despliegue en máquina virtual Linux

Integrantes	David Felipe Bejarano Montes [Nombre del integrante 2]
Docente	Ing., Esp., M.Ed. Christian Felipe Duarte
Asignatura	Bases de Datos 2 — Proyecto Final
Entregable	Fase 1 — Documento de propuesta

Bogotá D.C., octubre de 2026

Control del documento

Tabla 1. Datos de control

Título	SIRAV — Sistema Integral de Reservas y Alquiler de Vehículos
Versión	1.0
Fecha	17 de octubre de 2026
Estado	Propuesta para revisión
Asignatura	Bases de Datos 2 — Proyecto Final
Docente	Ing., Esp., M.Ed. Christian Felipe Duarte
Autores	David Felipe Bejarano Montes; [integrante 2]

Resumen ejecutivo

SIRAV es un prototipo académico de aplicación web para una empresa pequeña de alquiler de vehículos. Centraliza clientes, vehículos, reservas, alquileres, devoluciones e inspecciones, y evita por diseño que un mismo vehículo quede asignado a dos períodos que se superponen. La regla se garantiza en la base de datos, no solo en la interfaz.

La solución combina PostgreSQL, que concentra los datos transaccionales y la lógica de negocio (procedimientos, funciones, triggers, excepciones y transacciones), con MongoDB, que guarda las inspecciones y la bitácora de eventos. La aplicación se desarrolla con Spring Boot y se despliega sobre Ubuntu Server en una máquina virtual. El proyecto se gestiona bajo una metodología híbrida, con cronograma de cuatro semanas, matriz de riesgos, matriz RACI y control de versiones en GitHub.

Tabla 2. Ficha del proyecto

Tipo de proyecto	Aplicación web académica (prototipo funcional con datos de prueba)
Equipo	2 integrantes
Duración	4 semanas
Aplicación	Java, Spring Boot, Spring Security, Thymeleaf y Bootstrap
Base relacional	PostgreSQL (PL/pgSQL)
Base documental	MongoDB
Plataforma	Ubuntu Server en máquina virtual

Tabla de contenido

1. Introducción.... 4
1.1 Descripción del proyecto.... 4
1.2 Planteamiento del problema.... 4
1.3 Justificación.... 4
2. Objetivos.... 4
2.1 Objetivo general.... 4
2.2 Objetivos específicos.... 4
3. Alcance, supuestos y límites.... 5
3.1 Alcance y supuestos.... 5
3.2 Límites del prototipo.... 5
4. Beneficios esperados.... 5
5. Descripción funcional.... 6
5.1 Módulos principales.... 6
5.2 Recorrido de una operación.... 6
5.3 Ejemplo de uso.... 6
5.4 Reglas de negocio.... 7
6. Arquitectura y diseño de datos.... 7
6.1 Arquitectura propuesta.... 7
6.2 Modelo relacional.... 8
6.3 Modelo documental (MongoDB).... 8
6.4 Lógica de negocio en la base de datos.... 9
6.5 Seguridad e integridad.... 9
6.6 Reportes SQL.... 10
6.7 Virtualización y selección tecnológica.... 10
7. Gestión del proyecto.... 10
7.1 Metodología.... 10
7.2 Requerimientos.... 10
7.3 Gestión de riesgos.... 12
7.4 Control de versiones.... 12
7.5 Cronograma.... 13
7.6 Organización del equipo.... 13
8. Entregables.... 14
9. Demostración final prevista.... 14
10. Resultado esperado.... 15
Referencias.... 15

1. Introducción

1.1 Descripción del proyecto

SIRAV es una plataforma web diseñada para digitalizar las operaciones básicas de una empresa pequeña de alquiler de vehículos. Un cliente podrá consultar el catálogo, seleccionar fechas y solicitar una reserva. El personal administrativo podrá verificar disponibilidad, confirmar alquileres, registrar entregas y devoluciones, y visualizar información sobre el uso de la flota.

La propuesta se enfoca en un proceso completo y demostrable, desde la reserva hasta el cierre del alquiler. El sistema será un prototipo académico funcional con datos de prueba, no una plataforma comercial lista para operar con pagos o contratos jurídicos reales. Integra una base de datos relacional con lógica de negocio en el motor, una base documental MongoDB y un despliegue en una máquina virtual Linux, tal como lo exige el proyecto final de Bases de Datos 2.

1.2 Planteamiento del problema

Cuando las reservas se gestionan en hojas de cálculo, mensajes y registros manuales, pueden producirse asignaciones duplicadas, confusión sobre el estado de los automóviles, errores al liquidar el alquiler y dificultades para consultar el historial de cada vehículo. La falta de una fuente única de información también retrasa la atención al cliente y la elaboración de reportes.

1.3 Justificación

El control de reservas que no se superponen es un caso representativo de integridad y concurrencia en bases de datos: exige restricciones declarativas, transacciones, manejo de excepciones y pruebas con accesos simultáneos. Además, la información del negocio tiene dos naturalezas distintas (datos estructurados y transaccionales, y registros documentales de estructura variable), lo que justifica el uso complementario de PostgreSQL y MongoDB y permite aplicar de forma integrada los contenidos de Bases de Datos 1 y 2.

2. Objetivos

2.1 Objetivo general

Centralizar la información de clientes, vehículos y alquileres mediante una aplicación web que controle la disponibilidad, registre los movimientos de forma confiable y conserve un historial consultable, combinando PostgreSQL y MongoDB sobre un entorno Linux virtualizado y bajo una gestión formal de proyecto.

2.2 Objetivos específicos

- **Modelado.** Diseñar el modelo E-R (Chen), el E-R extendido, el modelo relacional en notación Crow's Foot y el diccionario de datos, normalizados hasta 3FN.
- **Lógica en la base de datos.** Implementar procedimientos almacenados, funciones, triggers, manejo de excepciones y control de transacciones en PostgreSQL (PL/pgSQL).
- **Base documental.** Implementar en MongoDB un modelo documental con CRUD, consultas complejas y agregaciones, justificando qué datos se almacenan allí.

- **Integración y despliegue.** Conectar una aplicación Spring Boot a ambas bases de datos y desplegarla en una máquina virtual Ubuntu Server documentada.
- **Gestión.** Planificar y controlar el proyecto con cronograma, requerimientos, riesgos, metodología y control de versiones en GitHub.

3. Alcance, supuestos y límites

3.1 Alcance y supuestos

- **Flota y sede.** Una sola sede y una flota de prueba de 20 a 30 vehículos en 4 o 5 categorías.
- **Reserva.** Cada reserva corresponde a un cliente y a un vehículo; no hay reservas de varios vehículos.
- **Dinero.** Tarifas, pagos y cargos son valores de demostración en pesos colombianos; no hay pasarela de pago.
- **Evidencias.** Las fotografías de inspección se referencian por nombre de archivo; no se almacenan binarios en la base.
- **Usuarios.** Se crean usuarios de prueba para los tres perfiles: administrador, operador y cliente.
- **Equipo.** El proyecto lo desarrollan dos integrantes; el alcance y las estimaciones se dimensionaron para esa capacidad.

3.2 Límites del prototipo

Para cumplir el plazo de cuatro semanas se excluyen pagos bancarios reales, geolocalización en tiempo real, firma electrónica con validez jurídica, verificación externa de licencias, servicios comerciales de seguros, una API REST con interfaz independiente y la replicación o fragmentación de MongoDB. Estos elementos pueden plantearse como ampliaciones futuras sin afectar el funcionamiento central de la solución.

4. Beneficios esperados

- **Control de disponibilidad.** Evitar reservas superpuestas para un mismo vehículo y período, incluso con accesos simultáneos.
- **Seguimiento del alquiler.** Conocer cuáles vehículos están disponibles, reservados, entregados o en mantenimiento.
- **Trazabilidad.** Consultar reservas, pagos de demostración, devoluciones, inspecciones e incidencias.
- **Apoyo operativo.** Generar reportes de utilización, ingresos simulados y estado de la flota.

5. Descripción funcional

5.1 Módulos principales

Tabla 3. Módulos del sistema

Módulo	Funcionalidad
Usuarios y perfiles	Inicio de sesión y diferenciación entre administrador, operador y cliente de prueba.
Catálogo de vehículos	Consulta de vehículos por categoría, características, tarifa diaria, estado y fechas libres.
Reservas	Selección de fechas, validación de disponibilidad y confirmación o cancelación.
Alquileres	Registro de entrega, fechas efectivas, inspección y condiciones de devolución.
Mantenimiento	Registro de períodos en los que un vehículo no puede reservarse.
Cobros simulados	Estimación del costo, registro de pagos de prueba y cargos adicionales por retraso.
Historial y reportes	Consulta de alquileres, ocupación de vehículos, incidencias y reportes SQL y MongoDB.

5.2 Recorrido de una operación

Tabla 4. Etapas del ciclo de reserva y alquiler

Paso	Etapa	Descripción
01	Consulta	El cliente elige una categoría o vehículo y revisa las fechas disponibles (fn_vehiculo_disponible).
02	Reserva	sp_crear_reserva verifica conflictos y crea la reserva en estado SOLICITADA.
03	Confirmación	Un operador valida la solicitud con sp_confirmar_reserva; el estado pasa a CONFIRMADA.
04	Entrega	sp_iniciar_alquiler crea el alquiler (estado ACTIVA) y la aplicación guarda la inspección inicial en MongoDB.
05	Devolución	Se registra el retorno y la inspección final; sp_cerrar_alquiler calcula cargos y genera el pago de prueba.
06	Cierre	La reserva queda CERRADA, el vehículo vuelve a DISPONIBLE y el historial queda consultable.

5.3 Ejemplo de uso

Un cliente solicita un automóvil compacto del 10 al 12 de noviembre. Antes de confirmar, el sistema comprueba que el vehículo no tenga una reserva vigente que coincida con ese intervalo. Si está libre, registra la solicitud; si ya está ocupado, informa que debe elegir otro vehículo o cambiar las fechas. Cuando el operador entrega el automóvil, la reserva pasa a alquiler activo. Al devolverlo, se calcula el valor de demostración y se cierra la operación.

5.4 Reglas de negocio

La regla más importante es que dos operaciones vigentes no puedan asignar el mismo vehículo para períodos que se superponen. Esta validación se implementará en la base de datos mediante una restricción de exclusión de PostgreSQL (rango de fechas con btree_gist) y no dependerá únicamente de la interfaz. El conjunto completo de reglas es el siguiente.

Tabla 5. Reglas de negocio

ID	Regla
RN-01	Un vehículo no puede tener dos reservas en estado SOLICITADA, CONFIRMADA o ACTIVA con períodos que se solapen.
RN-02	Un vehículo no puede reservarse en fechas que coincidan con un mantenimiento registrado.
RN-03	El estado de la reserva solo avanza en orden: SOLICITADA, CONFIRMADA, ACTIVA y CERRADA. CANCELADA solo es posible desde SOLICITADA o CONFIRMADA.
RN-04	El costo se calcula con la tarifa diaria vigente de la categoría; cada día de retraso en la devolución genera un cargo adicional del 50 % de esa tarifa (valor de demostración).
RN-05	Un alquiler solo se cierra después de registrar la devolución y su inspección.

6. Arquitectura y diseño de datos

6.1 Arquitectura propuesta

La aplicación utilizará Java con Spring Boot y páginas Thymeleaf/Bootstrap para presentar el catálogo y los formularios. PostgreSQL concentrará las operaciones estructuradas y transaccionales; MongoDB almacenará registros documentales como inspecciones y bitácoras. La aplicación se ejecutará en un servidor Ubuntu virtualizado y se accederá a ella desde un navegador.

Tabla 6. Capas de la arquitectura

Capa	Tecnología	Responsabilidad
Presentación	Thymeleaf + Bootstrap	Interfaz web para clientes y personal.
Aplicación	Java + Spring Boot, Spring Security	Autenticación, validación de entradas e invocación de procedimientos (SimpleJdbcCall/JdbcTemplate) y de MongoDB (Spring Data MongoDB).
Datos relacionales	PostgreSQL / PL/pgSQL	Reservas, disponibilidad, alquileres, tarifas, pagos de prueba, auditoría y lógica de negocio.
Datos documentales	MongoDB	Inspecciones, observaciones, evidencias referenciadas y bitácora de eventos.
Infraestructura	Ubuntu Server en VM	Alojamiento de la aplicación y de ambas bases de datos.

6.2 Modelo relacional

El diseño relacional incluirá 13 entidades: Usuario, Rol, Usuario_Rol, Cliente, Categoria_Vehiculo, Vehiculo, Tarifa, Reserva, Alquiler, Pago, Cargo_Adicional, Mantenimiento y Auditoria_Reserva. Se normalizará hasta 3FN: la tarifa depende de la categoría y de su vigencia, y el costo y la condición de «reservado» se calculan a partir de las reservas en lugar de duplicarse. Las relaciones cubren los tres tipos exigidos.

Tabla 7. Relaciones del modelo relacional

Tipo	Relación	Nota
1:1	Reserva – Alquiler	Un alquiler nace de exactamente una reserva (id_reserva único en Alquiler).
1:1	Usuario – Cliente	Un cliente con acceso web tiene un único usuario (relación opcional).
1:N	Categoria – Vehiculo; Categoria – Tarifa	Cada categoría agrupa varios vehículos y tiene tarifas con vigencia.
1:N	Cliente – Reserva; Vehiculo – Reserva; Vehiculo – Mantenimiento; Alquiler – Pago; Alquiler – Cargo_Adicional	Historial de cada cliente, vehículo y alquiler.
N:M	Usuario – Rol	Resuelta con la tabla Usuario_Rol.

6.3 Modelo documental (MongoDB)

MongoDB tendrá las colecciones inspecciones_vehiculo y bitacora_operaciones. Una inspección incluirá el identificador del alquiler, el tipo (entrega o devolución), fecha, combustible, kilometraje, una lista variable de daños (zona, severidad, descripción) y las evidencias referenciadas. La bitácora guardará eventos de la aplicación, como inicios de sesión e intentos de reserva rechazados.

Justificación del uso de NoSQL. Las inspecciones tienen estructura variable (cada una puede tener cero o muchos daños y evidencias) y se consultan por documento completo, algo incómodo de modelar en tablas; la bitácora es de escritura continua y sin relaciones. En cambio, el estado oficial de las reservas, los pagos y la auditoría transaccional permanecen en PostgreSQL como única fuente de verdad. El vínculo entre bases es el id_alquiler, que la aplicación verifica (no existen llaves foráneas entre motores).

Operaciones previstas. Se aplicarán CRUD completo (registrar, consultar, corregir y anular inspecciones), una consulta compleja (inspecciones con daños de severidad alta en un rango de fechas, con $elemMatch) y tres agregaciones: kilometraje y combustible promedio por categoría, zonas de daño más frecuentes por mes ($unwind y $group) e incidencias por vehículo.

6.4 Lógica de negocio en la base de datos

Tabla 8. Objetos de base de datos previstos

Objeto	Nombre	Función
Procedimientos	sp_crear_reserva, sp_confirmar_reserva, sp_iniciar_alquiler, sp_cerrar_alquiler, sp_cancelar_reserva	Cada uno ejecuta una operación completa y controlada; la aplicación solo los invoca.
Funciones	fn_vehiculo_disponible, fn_calcular_costo, fn_calcular_cargo_retraso	Consultan disponibilidad y calculan el valor según días, tarifa y retraso.
Triggers	trg_validar_transicion_reserva, trg_validar_mantenimiento, trg_auditar_reserva, trg_actualizar_estado_vehiculo	Protegen RN-02 y RN-03, registran cambios en Auditoria_Reserva y sincronizan el estado del vehículo.
Excepciones	e_reserva_solapada, e_vehiculo_en_mantenimiento, e_transicion_invalida	Errores propios con mensajes claros; la aplicación los traduce a avisos al usuario.
Transacciones	Dentro de cada sp_	La operación se confirma completa (COMMIT) o se revierte (ROLLBACK) si aparece un conflicto o error.

6.5 Seguridad e integridad

- **Integridad.** Llaves primarias y foráneas, UNIQUE (por ejemplo, la placa), CHECK (fecha_fin posterior a fecha_inicio, tarifas positivas) y la restricción de exclusión de RN-01.
- **Privilegios en PostgreSQL.** Un rol de aplicación con permiso de ejecutar los sp_ y consultar vistas, sin DELETE directo sobre reservas; un rol de solo lectura para reportes y un rol administrador.
- **Autenticación.** Contraseñas con hash (BCrypt) y autorización por perfil en Spring Security; usuario de MongoDB limitado a su base.
- **Rendimiento.** Índices (ix_) sobre vehículo y fechas de reserva, y sobre consultas frecuentes de reportes.

6.6 Reportes SQL

Tabla 9. Reportes SQL previstos

Reporte	Técnica SQL
R1. Ocupación de cada vehículo por mes (días alquilados frente a días del mes)	JOIN, GROUP BY, generate_series
R2. Ingresos simulados por categoría y por mes	Agregaciones y vista
R3. Los cinco vehículos más alquilados de cada categoría	Función de ventana RANK() OVER
R4. Clientes con más reservas y su tasa de cancelación	Subconsultas y CASE
R5. Devoluciones tardías y cargos adicionales	JOIN y HAVING
R6. Vehículos disponibles en un rango de fechas	NOT EXISTS / fn_vehiculo_disponible

6.7 Virtualización y selección tecnológica

Análisis de selección. Se compararán Ubuntu Server LTS, Debian y Rocky Linux (soporte, documentación, paquetes de PostgreSQL y MongoDB, consumo de recursos) y, para el motor relacional, PostgreSQL frente a MySQL/MariaDB (restricciones de exclusión, tipos de rango, PL/pgSQL y control transaccional). La hipótesis de trabajo es Ubuntu Server LTS con PostgreSQL, y se confirmará con una matriz de criterios ponderados.

Máquina virtual. Se creará una máquina virtual con Ubuntu Server. En su interior estarán PostgreSQL, MongoDB y la aplicación Spring Boot, administrados como servicios (systemd). Se configurará la red para ingresar desde el navegador del equipo anfitrión y se documentarán los servicios, puertos (aplicación, PostgreSQL y MongoDB), proceso de arranque y copias de respaldo (pg_dump y mongodump). Para el prototipo se recomienda reservar aproximadamente 6 GB de RAM, 2 núcleos virtuales y 40 GB de almacenamiento si el equipo físico lo permite; el perfil mínimo es de 4 GB.

7. Gestión del proyecto

7.1 Metodología

Se usará una metodología híbrida. La Fase 1 (modelado) sigue un enfoque secuencial tipo cascada, porque el modelo E-R, el modelo relacional y el diccionario deben cerrarse antes de implementar. La Fase 2 usa Scrum ligero: sprints de una semana, tablero Kanban en GitHub Projects y revisión conjunta al terminar cada semana. El trabajo se gestiona en Jira con un tablero Scrum de cuatro sprints semanales (SIRAV Sprint 1 a 4), cada uno con su meta; un integrante actúa como Product Owner y el otro como Scrum Master, y los roles pueden rotar. Se eligió porque el plazo es corto, el alcance está definido y se necesita detectar bloqueos a tiempo.

7.2 Requerimientos

La prioridad se expresa como Alta (indispensable), Media (importante) o Baja (deseable). La última columna da la trazabilidad hacia el objeto que implementa cada requerimiento.

Tabla 10. Requerimientos funcionales (RF) y no funcionales (RNF)

ID	Requerimiento	Prioridad	Trazabilidad
RF-01	Autenticación y perfiles (administrador, operador, cliente).	Alta	Usuario, Rol; Spring Security
RF-02	Consultar el catálogo por categoría y fechas libres.	Alta	fn_vehiculo_disponible; R6
RF-03	Crear y cancelar reservas sin solapamientos.	Alta	sp_crear_reserva; exclusión RN-01
RF-04	Confirmar reservas e iniciar alquileres.	Alta	sp_confirmar_reserva; sp_iniciar_alquiler
RF-05	Registrar devolución, cargos y pago de prueba.	Alta	sp_cerrar_alquiler; fn_calcular_costo
RF-06	Registrar inspecciones de entrega y devolución.	Alta	MongoDB: inspecciones_vehiculo
RF-07	Consultar historial y reportes.	Media	R1 a R6; agregaciones MongoDB
RF-08	Registrar mantenimientos de vehículos.	Media	Mantenimiento; trg_validar_mantenimiento
RNF-01	Sin dobles asignaciones, aun con accesos simultáneos.	Alta	Restricción de exclusión; prueba de concurrencia
RNF-02	Seguridad: contraseñas con hash y privilegios mínimos.	Alta	Roles de PostgreSQL; BCrypt
RNF-03	Despliegue reproducible y documentado en VM Linux.	Alta	Guía de instalación y scripts
RNF-04	Consultas frecuentes en menos de 2 segundos con los datos de prueba.	Media	Índices ix_

7.3 Gestión de riesgos

Tabla 11. Matriz de riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R1	Curva de aprendizaje de restricciones de exclusión y PL/pgSQL.	Media	Alto	Prototipo en la semana 1; alternativa con trigger y bloqueo de filas (SELECT FOR UPDATE).
R2	Recursos insuficientes del equipo para la VM.	Media	Alto	Perfil mínimo de 4 GB; probar la VM en la semana 1.
R3	Retraso al integrar Spring Boot con dos bases de datos.	Media	Alto	Probar ambas conexiones en la semana 2, antes de construir la interfaz.
R4	Reparto desigual de trabajo o integrante no disponible (equipo de dos).	Media	Alto	Revisión semanal, puntos de historia por persona en Jira y respaldo cruzado de roles.
R5	Crecimiento del alcance con funciones nuevas.	Alta	Medio	Alcance congelado; lo adicional pasa a ampliaciones futuras.
R6	Pérdida de código o de la máquina virtual.	Baja	Alto	Repositorio en GitHub, scripts de despliegue, instantánea de la VM y copias de las bases.

La matriz gráfica de probabilidad por impacto se incluirá en el documento de gestión del proyecto.

7.4 Control de versiones

- **Repositorio.** GitHub, con scripts SQL numerados, código de la aplicación, documentación y guía de la VM.
- **Ramas.** main (estable), develop (integración) y ramas feature/ por tarea; la integración se hace con pull requests revisados por otro integrante.
- **Commits.** Mensajes descriptivos con prefijo (feat, fix, docs, db) y referencia a la tarea.
- **Equidad.** Cada integrante hace commits en su área; el historial y las estadísticas de contribución sirven de evidencia.

7.5 Cronograma

Tabla 12. Cronograma de cuatro semanas

Semana	Trabajo central	Resultado e hito
1	Propuesta, supuestos, requerimientos, riesgos, Gantt y RACI; modelos E-R (Chen y extendido), Crow's Foot y relacional; diccionario de datos; análisis de selección de SO y RDBMS; instalación de la VM.	H1: modelado cerrado, gestión definida y entorno inicial listo.
2	Scripts de creación e inserción; procedimientos, funciones, triggers, excepciones y transacciones; roles; documento SQL de reportes; modelo documental de MongoDB.	H2: base relacional probada y entregables de la Fase 1 completos.
3	Implementación de MongoDB (CRUD, consultas complejas, agregaciones); catálogo, reservas y alquileres en la aplicación; conexión a ambas bases.	H3: versión web integrada para pruebas.
4	Pruebas de concurrencia y validaciones; reportes en la aplicación; informe ejecutivo (máximo 5 páginas) y referencias; video de máximo 15 minutos; repositorio final.	H4: prototipo verificable y entrega final.

Cada semana corresponde a un sprint de Jira con su meta y su hito. Las fechas oficiales de cada fase se ajustarán al calendario del curso. El diagrama de Gantt detallado, con tareas y responsables, acompañará el documento de gestión.

7.6 Organización del equipo

Tabla 13. Roles y responsabilidades

Integrante	Rol	Responsabilidades principales
Integrante 1	Product Owner; datos relacionales y calidad	Modelos E-R y relacional, procedimientos, funciones y restricción de exclusión, flujos de reservas y alquileres en la aplicación, pruebas de concurrencia e informe ejecutivo.
Integrante 2	Scrum Master; MongoDB e infraestructura	Diccionario de datos, triggers, reportes SQL y roles, máquina virtual y despliegue, MongoDB, base de la aplicación y catálogo, pruebas end-to-end y video.

Ambos integrantes participan en la definición del flujo de alquiler, el repositorio GitHub, la integración, las pruebas finales, el video y la sustentación. La matriz RACI resume las responsabilidades: R = ejecuta, A = responde por el resultado, C = consultado.

Tabla 14. Matriz RACI

Actividad	Integrante 1	Integrante 2
Modelado y diccionario de datos	A/R	R
Procedimientos, funciones y restricción de exclusión	A/R	C
Triggers, reportes SQL y roles	C	A/R
Aplicación web: flujos y conexiones	A/R	R
MongoDB (modelo, CRUD, agregaciones)	C	A/R
Máquina virtual y despliegue	C	A/R
Jira, Gantt, requerimientos y riesgos	A/R	R
Pruebas de concurrencia y end-to-end	A/R	R
Informe ejecutivo, video y sustentación	A/R	R

8. Entregables

Tabla 15. Entregables por fase

Fase	Entregables
Fase 1	Documento de propuesta, documento de supuestos, modelo E-R (Chen), E-R extendido, E-R y modelo relacional en Crow's Foot (Data Modeler o Workbench), diccionario de datos y documento SQL de reportes.
Fase 2	Análisis de selección de SO y de RDBMS, VM implementada, RDBMS implementado, scripts de creación, inserción y varios, modelo documental e implementación en MongoDB, conexiones de la aplicación a ambas bases, repositorio Git y video demostrativo.
Cierre	Informe ejecutivo de máximo 5 páginas, con problema, solución, arquitectura, decisiones técnicas, resultados, lecciones aprendidas y conclusiones.

9. Demostración final prevista

La sustentación (video de máximo 15 minutos, con participación de ambos integrantes) iniciará mostrando un automóvil disponible y una reserva creada correctamente con sp_crear_reserva. Después se intentará registrar otra reserva con fechas superpuestas, para mostrar su rechazo por la base de datos y el mensaje de excepción en la aplicación. A continuación, el operador entregará el vehículo, registrará una inspección en MongoDB, cerrará el alquiler, consultará su cobro simulado y revisará el historial de la operación, junto con una agregación de MongoDB y un reporte SQL. Finalmente se mostrará que la aplicación y las dos bases de datos están instaladas en Ubuntu Server dentro de la máquina virtual, y se repasarán el repositorio y las evidencias de gestión.

10. Resultado esperado

Al finalizar el mes se espera contar con una aplicación navegable que registre el ciclo de reserva y alquiler de vehículos, mantenga datos consistentes, permita consultas operativas y demuestre el uso complementario de PostgreSQL y MongoDB sobre un entorno Linux virtualizado, acompañada de la documentación y la gestión de proyecto que exige la asignatura.

Referencias

Bradshaw, S., Brazil, E., & Chodorow, K. (2019). MongoDB: The definitive guide (3.ª ed.). O'Reilly Media.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Sadalage, P. J., & Fowler, M. (2012). NoSQL distilled: A brief guide to the emerging world of polyglot persistence. Addison-Wesley.

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum. https://scrumguides.org

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7.ª ed.). McGraw-Hill.

Esta lista se ampliará con artículos de revistas científicas en el informe final.